Chapter 5. Numbers

5.0 Introduction

Numbers are basic to just about any computation. They're used for array indices, temperatures, salaries, ratings, and an infinite variety of things. Yet they're not as simple as they seem. With floating-point numbers, how accurate is accurate? With random numbers, how random is random? With strings that should contain a number, what actually constitutes a number?

Java has eight built-in, or *primitive*, types, seven of which can be used to represent numbers. These are summarized in Table 5-1 with their *wrapper* (object) types, as well as two numeric classes that do not represent primitive types. The eighth primitive, `boolean`, and its wrapper `Boolean`, aren't numeric and can't be treated as such. Note that unlike languages such as C or Perl, which don't specify the size or precision of numeric types, Java—with its goal of portability—specifies these exactly and states that they are the same on all platforms.

Table 5-1. Numeric types

Built-in type	Object wrapper	Size of built-in (bits)	Contents
`byte`	`Byte`	8	Signed integer
`short`	`Short`	16	Signed integer
`int`	`Integer`	32	Signed integer
`long`	`Long`	64	Signed integer
`float`	`Float`	32	IEEE-754 floating point (single precision)
`double`	`Double`	64	IEEE-754 floating point (double precision)
`char`	`Character`	16	Unsigned Unicode character
n/a	`BigInteger`	unlimited	Arbitrary-size immutable integer value
n/a	`BigDecimal`	unlimited	Arbitrary-size-and-precision immutable floating-point value

As you can see, Java provides a numeric type for just about any purpose. There are four sizes of signed integers for representing various sizes of whole numbers. There are two sizes of floating-point numbers to approximate real numbers. There is also a type specifically designed to represent and allow operations on Unicode characters. The primitive numeric types are discussed here. The `Big` value types are described in Recipe 5.13.

When you read a string representing a number from user input or a text file, you need to convert it to the appropriate type. The object wrapper classes in the second column have several functions, one of which is to provide this basic conversion functionality—replacing the C programmer's `atoi` / `atof` family of functions and the numeric arguments to `scanf`.

Going the other way, you can convert any number to a string just by using string concatenation, or by using the `static` method `toString(value)` in each of the wrapper classes. If you want full control, Recipe 5.5 shows the use of `NumberFormat` and its related classes to provide full control of formatting.

As the name *object wrapper* implies, these classes are also used to wrap a number in a Java object, as many parts of the standard API are defined in terms of objects. Later on, "Solution" shows using an `Integer` object to save an `int`'s value to a file using object serialization and retrieving the value later.

But I haven't yet mentioned the issues of floating point. Real numbers, you may recall, are numbers with a fractional part. There is an infinite number of real numbers. A floating-point number—what a computer uses to approximate a real number—is not the same as a real number. The number of floating-point numbers is finite, with only 2^32 different bit patterns for `float`s, and 2^64 for `double`s. Thus, most real values have only an approximate correspondence to floating point. The result of printing the real number 0.3 works correctly, like this:

```
// numbers/RealValues.java
System.out.println("The real value 0.3 is " + 0.3);
```

That code results in this printout:

```
The real value 0.3 is 0.3
```

But the difference between a real value and its floating-point approximation can accumulate if the value is used in a computation; this is often called a *rounding error*. Continuing the previous example, the real 0.3 multiplied by 3 yields:

```
The real 0.3 times 3 is 0.89999999999999991
```

Surprised? Not only is it off by a bit from what you might expect, but you will get the same output on any conforming Java implementation. I ran it on machines as disparate as an AMD/Intel PC with OpenBSD, a PC with Windows and the standard JDK, and on macOS, in Java versions from 8 to 22. Always the same answer. Whereas 3.0 * 3 gives 9.0, as you'd expect. And the same occurs in the C programming language, and should occur in any language that conforms to the IEEE Standard 754 for floating-point computation.

And what about random numbers? How random are they? You have probably heard the term *pseudorandom number generator*, or PRNG. All conventional random number generators, whether written in Fortran, C, or Java, generate pseudorandom numbers. That is, they're not truly random! True randomness comes only from specially built hardware: an analog source of Brownian noise connected to an analog-to-digital converter, for example.[1] Your average PC of today may have some good sources of entropy, or even hardware-based sources of randomness (which have not been widely used or tested yet). However, pseudorandom number generators are good enough for most purposes (other than cryptography, where special care is needed). Java provides one random generator in the base library java.lang.Math, and several others; we'll examine these in Recipe 5.9.

The class `java.lang.Math` contains an entire math library in one class, including trigonometry, conversions (including degrees to radians and back), rounding, truncating, square root, minimum, and maximum. It's all there. Check the Javadoc for details on the math library.

The package java.math contains support for *big numbers*—those larger than the normal built-in long integers, for example. See Recipe 5.13.

Java works hard to ensure that your programs are reliable. The usual ways you'd notice this are in the common requirement to catch potential exceptions—all through the Java API—and in the need to *cast*, or convert, when storing a value that might or might not fit into the variable you're trying to store it in. I'll show examples of these.

Overall, Java's handling of numeric data fits well with the ideals of portability, reliability, and ease of programming.

See Also

The Java Language Specification, and the Javadoc page for `java.lang.Math`.

5.1 Checking Whether a String Is a Valid Number

Problem

You need to check whether a given string contains a valid number, and, if so, convert it to binary (internal) form.

Solution

To accomplish this, either use the appropriate wrapper class's conversion routine and catch the `NumberFormatException`, or try to match with a regular expression (Chapter 4). The code in Example 5-1 converts a string to a `double`.

Example 5-1. main/src/main/java/numbers/StringToDouble.java

```
public static void main(String[] argv) {
  String aNumber = argv[0];  // not argv[1]
  double result;
  try {
```

```
    result = Double.parseDouble(aNumber);
    System.out.println("Number is " + result);
  } catch(NumberFormatException exc) {
    System.out.println("Invalid number " + aNumber);
    return;
  }
}
```

Discussion

This code (with suitable change for any other numeric wrapper class) lets you validate numbers in the format that the designers of the wrapper classes expected. If you need to accept a different definition of numbers, you could use regular expressions (see Chapter 4) to make the determination.

There may also be times when you want to tell whether a given numeric string contains an integer number or a floating-point number. One way is to check for the characters `.`, `d`, `e`, or `f` in the input; if one of these characters is present, convert the number as a `double`. Otherwise, convert it as an `int`, as in Example 5-2. Another possibility is to convert it to a `Double` (using `valueOf(String)`), then test `myDouble.intValue()==myDouble.doubleValue()`, which should always be true if there is no decimal fraction.

Example 5-2. main/src/main/java/numbers/GetNumber.java

```
/*
 * Process one String, displaying it as int or double as appropriate
 */
public static Number process(String s) {
  if (s.matches("[+-]*\\d*\\.\\d+[dDeEfF]*")) {
    try {
      double dValue = Double.parseDouble(s);
      System.out.println(s + ": is a double: " + dValue +
        " by Double.parseDouble()");
      return dValue;
    } catch (NumberFormatException e) {
      System.out.println(s + ": Invalid double: " + s);
      return Double.NaN;
    }
  } else try {
    // did not contain . d e or f, so try as long.
      long longValue = Long.parseLong(s);
      System.out.println(s + ": is a long: " + longValue +
        " by Long.parseLong()");
      return longValue;
    } catch (NumberFormatException e2) {
      System.out.println(s + ": Not a number: " + s);
      return Double.NaN;
    }
}

public boolean isIntegerNumber(String s) {
  Double d = Double.valueOf(s);
  boolean result = d.intValue() == d.doubleValue();
  System.out.println(s + ": is " + (result ?
    "an integer" : "a floating-point number") +
    " by Double.xxxValue() comparison");
  return result;
}
```

See Also

A more involved form of parsing is offered by the `DecimalFormat` class, discussed in Recipe 5.5. There is also the `Scanner` class; see Recipe 10.7.

5.2 Converting Numbers to Objects and Vice Versa

Problem

You need to convert numbers to objects and objects to numbers.

Solution

Use the object wrapper classes listed in Table 5-1 at the beginning of this chapter.

Discussion

Often you have a primitive number and you need to pass it into a method where an `Object` is required, or vice versa. Long ago you had to invoke the conversion routines that are part of the `wrapper` classes, but now you can generally use automatic conversion (called *auto-boxing*/*auto-unboxing*). See Example 5-3 for examples of both.

Example 5-3. main/src/main/java/structure/AutoboxDemo.java

```
public class AutoboxDemo {

    /** Shows auto-boxing (in the call to foo(i), i is wrapped automatically)
     * and auto-unboxing (the return value is automatically unwrapped).
     */
    public static void main(String[] args) {
        int i = 42;
        int result = foo(i);          ❶
        System.out.println(result);
    }

    public static Integer foo(Integer i) {
        System.out.println("Object = " + i);
        return Integer.valueOf(123);  ❷
    }
}
```

❶ Auto-boxing: `int 42` is converted to `Integer(42)`. Also auto-unboxing: the `Integer` returned from `foo()` is auto-unboxed to assign to `int result`.

❷ No auto-boxing: `valueOf()` returns `Integer`. If the line said `return Integer.intValueOf(123)`, then it would be a second example of auto-boxing because the method return value is `Integer`.

To explicitly convert between an `int` and an `Integer` object, or vice versa, you can use the `wrapper` class methods:

```
// int to Integer
Integer wrapped = Integer.valueOf(42);
System.out.println(wrapped.toString());    // or just "wrapped"

// Integer to int
int primitive = wrapped.intValue();
System.out.println(primitive);
```

TIP

Any boxed value of `boolean`, `byte`, `char` up to 127, and `short` or `int` from –128 to +127, will be *interned*, that is, only a single instance will exist no matter how many times you do the boxing conversion of a given value. This may give unexpected results when using `==` comparisons.

5.3 Taking a Fraction of an Integer Without Using Floating Point

Problem

You want to multiply an integer by a fraction without converting the fraction to a floating-point number.

Solution

Multiply the integer by the numerator and divide by the denominator.

This technique should be used only when efficiency is more important than clarity because it tends to detract from the readability—and therefore the maintainability—of your code.

Discussion

Because integers and floating-point numbers are stored differently, it may sometimes be desirable and feasible, for efficiency purposes, to multiply an integer by a fractional value without converting the values to floating point and back, and without requiring a cast. This is shown in Example 5-4.

Example 5-4. main/src/main/java/numbers/FractMult.java

```
public class FractMult {
  public static void main(String[] u) {

    double d1 = 0.666 * 5;  // fast but obscure and inaccurate: convert
    System.out.println(d1); // 2/3 to 0.666 in programmer's head

    double d2 = 2/3 * 5;    // wrong answer - 2/3 == 0, 0*5 = 0
    System.out.println(d2);

    double d3 = 2d/3d * 5;  // "normal"
    System.out.println(d3);

    double d4 = (2*5)/3d;   // one step done as integers, almost same answer
    System.out.println(d4);

    int i5 = 2*5/3;         // fast, approximate integer answer
    System.out.println(i5);
  }
}
```

Running the code looks like this:

```
$ java numbers.FractMult
3.33
0.0
3.333333333333333
3.3333333333333335
3
$
```

You should beware of the possibility of numeric overflow and avoid this optimization if you cannot guarantee that the multiplication by the numerator will not overflow.

5.4 Working with Floating-Point Numbers

Problem

You want to be able to compare and round floating-point numbers.

Solution

Compare with the `INFINITY` constants, and use `isNaN()` to check for `NaN` (not a number).

Compare floating values with an epsilon value.

Round floating-point values with `Math.round()` or custom code.

Discussion

Comparisons can be a bit tricky. Fixed-point (`short`, `int`, `long`) operations that divide by zero result in Java notifying you abruptly by throwing an exception. This is because integer division by zero is considered a *logic error*.

Floating-point operations, however, do not throw an exception because they are defined over an (almost) infinite range of values. Instead, they signal errors by producing the constant `POSITIVE_INFINITY` if you divide a positive floating-point number by zero, the constant `NEGATIVE_INFINITY` if you divide a negative floating-point value by zero, and `NaN` if you otherwise generate an invalid result. Values for these three public constants are defined in both the `Float` and the `Double` wrapper classes. The value `NaN` has the unusual property that it is not equal to itself (i.e., `NaN != NaN`). Thus, it would hardly make sense to compare a (possibly suspect) number against `NaN`, because the following expression can never be true:

```
x == NaN
```

Instead, the methods `Float.isNaN(float)` and `Double.isNaN(double)` must be used, as in Example 5-5.

Example 5-5. main/src/main/java/numbers/InfNaN.java

```
public static void main(String[] argv) {
  double d = 123;
  double e = 0;
  if (d/e == Double.POSITIVE_INFINITY)
    System.out.println("Check for POSITIVE_INFINITY works");
  double s = Math.sqrt(-1);
  if (s == Double.NaN)
    System.out.println("Comparison with NaN incorrectly returns true");
  if (Double.isNaN(s))
    System.out.println("Double.isNaN() correctly returns true");
}
```

Note that this, by itself, is not sufficient to ensure that floating-point calculations have been done with adequate accuracy. For example, the following program demonstrates a contrived calculation—Heron's formula for the area of a triangle—both in `float` and in `double`. The double values are correct, but the floating-point value comes out as zero due to rounding errors. This happens because, in Java, operations involving only `float` values are performed as 32-bit calculations. Related languages such as C automatically promote these to double during the computation, which can eliminate some loss of accuracy. Let's take a look:

```
public class Heron {
  public static void main(String[] args) {
    // Sides for triangle in float
    float af, bf, cf;
    float sf, areaf;

    // Ditto in double
    double ad, bd, cd;
    double sd, aread;

    // Area of triangle in float
    af = 12345679.0f;
    bf = 12345678.0f;
    cf = 1.01233995f;

    sf = (af+bf+cf)/2.0f;
    areaf = (float)Math.sqrt(sf * (sf - af) * (sf - bf) * (sf - cf));
    System.out.println("Single precision: " + areaf);
```

```
    // Area of triangle in double
    ad = 12345679.0;
    bd = 12345678.0;
    cd = 1.01233995;

    sd = (ad+bd+cd)/2.0d;
    aread = Math.sqrt(sd * (sd - ad) * (sd - bd) * (sd - cd));
    System.out.println("Double precision: " + aread);
  }
}
```

Now let's run it:

```
$ java numbers.Heron
Single precision: 0.0
Double precision: 972730.0557076167
$
```

If in doubt, use `double`!

NOTE

Prior to Java 17, there were potential issues around large-magnitude `double` computations that might give different results on different Java implementations. Java provided the keyword `strictfp`, which can apply to classes, interfaces, or methods within a class.[2] If a computation is Strict-FP, then it must always, for example, return the value `INFINITY` if a calculation would overflow the value of `Double.MAX_VALUE` (or underflow the value `Double.MIN_VALUE`). Non-Strict-FP calculations—formerly the default—were allowed to perform calculations on a greater range and can return a valid final result that is in range even if the interim product is out of range. This is pretty esoteric and affects only computations that approach the bounds of what fits into a double. Java 17 changed the default for all floating-point computations to Strict-FP; thus the `strictfp` keyword, while still allowed, no longer has any effect.

Comparing floating-point values

Based on what we've just discussed, you probably won't just go comparing two floats or doubles for equality. You might expect the floating-point wrapper classes, `Float` and `Double`, to override the `equals()` method, which they do. The `equals()` method returns `true` if the two values are the same bit for bit (i.e., if and only if the numbers are the same or are both `NaN`). It returns `false` otherwise, including if the argument passed in is null, or if one object is +0.0 and the other is -0.0.

I said earlier that `NaN != NaN`, but if you compare with `equals()`, the result is true:

```
jshell> Float f1 = Float.valueOf(Float.NaN)
f1 ==> NaN

jshell> Float f2 = Float.valueOf(Float.NaN)
f2 ==> NaN

jshell> f1 == f2 # Comparing object identities
$4 ==> false

jshell> f1.equals(f1) # bitwise comparison of values
$5 ==> true
```

If this sounds weird, remember that the complexity comes partly from the nature of doing real number computations in the less-precise floating-point hardware. It also comes partly from the details of the IEEE Standard 754, which specifies the floating-point functionality that Java tries to adhere to so that underlying floating-point processor hardware can be used even when Java programs are being interpreted.

To actually compare floating-point numbers for equality, it is generally desirable to compare them within some tiny range of allowable differences; this range is often regarded as a tolerance or as *epsilon*. Example 5-6 shows an `equals()` method you can use to do this comparison, as well as comparisons on values of `NaN`. When run, it prints that the first two numbers are equal within epsilon:

```
$ java numbers.FloatCmp
True within epsilon 1.0E-7
$
```

Example 5-6. main/src/main/java/numbers/FloatCmp.java

```
public class FloatCmp {

 final static double EPSILON = 0.0000001;

 public static void main(String[] argv) {
  double da = 3 * .3333333333;
  double db = 0.99999992857;

  // Compare two numbers that are expected to be close.
  if (da == db) {
   System.out.println("Java considers " + da + "==" + db);
  // else compare with our own equals overload
  } else if (equals(da, db, 0.0000001)) {
   System.out.println("Equal within epsilon " + EPSILON);
  } else {
   System.err.println(da + " != " + db);
  }

  System.out.println("NaN prints as " + Double.NaN);

  // Show that comparing two NaNs is not a good idea:
  double nan1 = Double.NaN;
  double nan2 = Double.NaN;
  if (nan1 == nan2)
   System.out.println("Comparing two NaNs incorrectly returns true.");
  else
   System.err.println("Comparing two NaNs correctly reports false.");

  if (Double.valueOf(nan1).equals(Double.valueOf(nan2)))
   System.out.println("Double(NaN).equals(NaN) correctly returns true.");
  else
   System.err.println("Double(NaN).equals(NaN) incorrectly returns false.");
 }

 /** Compare two doubles within a given epsilon */
 public static boolean equals(double a, double b, double eps) {
  if (a==b) return true;
  // If the difference is less than epsilon, treat as equal.
  return Math.abs(a - b) < eps;
 }

 /** Compare two doubles, using default epsilon */
 public static boolean equals(double a, double b) {
  return equals(a, b, EPSILON);
 }
}
```

Note that none of the `System.err` messages about incorrect returns prints. The point of this example with `NaN`s is that you should always make sure values are not `NaN` before entrusting them to `Double.equals()`.

Rounding

If you simply cast a floating value to an integer value, Java truncates the value. A value like 3.999999 cast to an `int` or `long` becomes 3, not 4. To round floating-point numbers properly, use `Math.round()`. It has two overloads: if you give it a `double`, you get a `long` result; if you give it a `float`, you get an `int`.

What if you don't like the rounding rules used by `round`? If, for some strange reason, you wanted to round numbers greater than 0.54 instead of the normal 0.5, you could write your own version of `round()`:

```
public class Round {
 /** We round a number up if its fraction exceeds this threshold. */
 public static final double THRESHOLD = 0.54;

 /*
  * Round floating values to integers.
  * @return the closest int to the argument.
  * @param d A non-negative value to be rounded.
  */
 public static int round(double d) {
  return (int)Math.floor(d + 1.0 - THRESHOLD);
 }

 public static void main(String[] argv) {
  for (double d = 0.1; d<=1.0; d+=0.05) {
   System.out.println("My way:  " + d + "-> " + round(d));
   System.out.println("Math way:" + d + "-> " + Math.round(d));
  }
 }
}
```

If, on the other hand, you simply want to display a number with less precision than it normally gets, you probably want to use a `DecimalFormat` object, which we look at in Recipe 5.5, or use a `printf`-like format code, discussed in Recipe 3.2.

5.5 Formatting Numbers

Problem

You need to format numbers.

Solution

Use a `NumberFormat` subclass. Or use a `Formatter`.

Discussion

The C language `printf()` function provides many formatting abilities. Java did not originally provide C-style `printf/scanf` functions because they tend to mix together formatting and input/output in a very inflexible way. Programs using `printf/scanf` can be hard to internationalize, for example. Of course, by popular demand, Java did eventually introduce `printf()`, which along with `String.format()` is now standard in Java, based on an underlying `Formatter` class. See Recipe 3.2.

Java has an entire package, java.text, full of formatting routines as general and flexible as anything you might imagine. As with `printf`, it has an involved formatting language, described in the Javadoc page. Consider the presentation of long numbers. In North America, the number "one thousand twenty-four and a quarter" is written 1,024.25. In other parts of the world it is written 1 024,25 or 1.024,25. Not to mention how currencies and percentages are formatted! Trying to keep track of this yourself would drive the average small software shop around the bend rather quickly.

Fortunately, the java.util package includes a `Locale` class; and, furthermore, the Java runtime automatically sets a default `Locale` object based on the user's environment (on Macintosh and Windows, the user's preferences, and on Unix, the user's environment variables).

To provide a nondefault locale in code, see Recipe 3.12. To provide formatters customized for numbers, currencies, and percentages, the `NumberFormat` class has static *factory methods* that normally return a `DecimalFormat` with the correct pattern already instantiated. A `DecimalFormat` object appropriate to the user's locale can be obtained from the factory method `NumberFormat.getInstance()` and manipulated using `set` methods. Surprisingly, the method `setMinimumIntegerDigits()` turns out to be the easy way to generate a number format with leading zeros. Example 5-7 demonstrates this.

Example 5-7. main/src/main/java/numbers/NumFormat2.java

```
public class NumFormat2 {
```

```
  public static final double data[] = {
    0, 1, 22d/7, 100.2345678
  };

  public static void main(String[] av) {
    // Get a format instance
    NumberFormat form = NumberFormat.getInstance();

    // Set it to look like 999.99[99]
    form.setMinimumIntegerDigits(3);
    form.setMinimumFractionDigits(2);
    form.setMaximumFractionDigits(4);

    // Now print using it.
    for (double d : data) {
      System.out.println(
        d + "\tformats as " + form.format(d));
    }
  }
}
```

This prints the contents of the array using the `NumberFormat` instance `form`:

```
$ java numbers.NumFormat2
0.0     formats as 000.00
1.0     formats as 001.00
3.142857142857143       formats as 003.1429
100.2345678     formats as 100.2346
$
```

You can also construct a `DecimalFormat` with a particular pattern or change the pattern dynamically using `applyPattern()`. Some of the more common pattern characters are shown in Table 5-2.

Table 5-2. `DecimalFormat` pattern characters

Character	Meaning
`#`	Numeric digit (leading zeros suppressed)
0	Numeric digit (leading zeros provided)
.	Locale-specific decimal separator (decimal point)
,	Locale-specific grouping separator (comma in English)
`-`	Locale-specific negative indicator (minus sign)
%	Shows the value as a percentage
`;`	Separates two formats: the first for positive and the second for negative values
'	Escapes one of the preceding characters so it appears
Anything else	Appears as itself

The `NumFormatDemo` program uses one `DecimalFormat` to print a number with only two decimal places and a second to format the number according to the default locale, as shown in Example 5-8.

Example 5-8. main/src/main/java/numbers/NumFormatDemo.java

```
  /** A number to format */
  public static final double intlNumber = 1024.25;
  /** Another number to format */
  public static final double ourNumber = 100.2345678;
    NumberFormat defForm = NumberFormat.getInstance();
    NumberFormat ourForm = new DecimalFormat("##0.##");
    // toPattern() will reveal the combination of #0., etc
    // that this particular Locale uses to format with!
    System.out.println("defForm's pattern is " +
```

```
    ((DecimalFormat)defForm).toPattern());
  System.out.println(intlNumber + " formats as " +
    defForm.format(intlNumber));
  System.out.println(ourNumber + " formats as " +
    ourForm.format(ourNumber));
  System.out.println(ourNumber + " formats as " +
    defForm.format(ourNumber) + " using the default format");
```

This program prints the given pattern and then formats the same number using several formats:

```
$ java numbers.NumFormatDemo
defForm's pattern is
,
0.
1024.25 formats as 1,024.25
100.2345678 formats as 100.23
100.2345678 formats as 100.235 using the default format
$
```

Human-readable number formatting

12 To print a number in what Linux/Unix calls "human-readable format" (many display commands accept a `-h` argument for this format), use the Java 12 `CompactNumberFormat`, as shown in Example 5-9.

Example 5-9. nmain/src/main/java/numbers/CompactFormatDemo.java

```
public class CompactFormatDemo {

  static final Number[] nums = {
    0, 1, 1.25, 1234, 12345, 123456.78, 123456789012L
  };
  static final String[] strs = {
    "1", "1.25", "1234", "12.345K", "1234556.78", "123456789012L"
  };

  public static void main(String[] args) throws ParseException {
    NumberFormat cnf = NumberFormat.getCompactNumberInstance();
    System.out.println("Formatting:");
    for (Number n : nums) {
      cnf.setParseIntegerOnly(false);
      cnf.setMinimumFractionDigits(2);
      System.out.println(n + ": " + cnf.format(n));
    }
    System.out.println("Parsing:");
    for (String s : strs) {
      System.out.println(s + ": " + cnf.parse(s));
    }
  }

}
```

The output is shown here:

```
$ java CompactFormatDemo.java
Formatting:
0: 0
1: 1
1.25: 1
1234: 1.23K
12345: 12.35K
123456.78: 123.46K
123456789012: 123.46B
Parsing:
1: 1
1.25: 1.25
```

```
1234: 1234
12.345K: 12345
1234556.78: 1234556.78
123456789012L: 123456789012
$
```

Roman numeral formatting

This is presented as a good example of writing your own `Format` subclass for unusual formatting needs; it's obviously not that common to need Roman numerals these days. To work with Roman numerals, use my `RomanNumberFormat` class, as in Example 5-10.

Example 5-10. main/src/main/java/numbers/RomanNumberSimple.java

```
RomanNumberFormat rf = new RomanNumberFormat();
var year = LocalDate.now().getYear();
var yearStr = rf.format(year);
System.out.println(year + " -> " + yearStr);
long newYear = (Long) rf.parseObject(yearStr);
System.out.println(yearStr + " -> " + newYear);
if (newYear != year) {
  System.out.println("Error: non-idempotent!");
}
```

Running `RomanNumberSimple` in 2025 produces this output:

```
2025 -> MMXXV
MMXXV -> 2025
```

The source of the `RomanNumberFormat` class, not included here to save space, is in *darwinsys-api/src/main/java/com/darwinsys/numbers/RomanNumberFormat.java* and online at my GitHub repository. Some of the public methods are required because I wanted it to be a subclass of the abstract class `Format`, including three different `format()` overloads.

See Also

Java I/O by Elliotte Rusty Harold (O'Reilly) includes an entire chapter on `NumberFormat` and develops the subclass `ExponentialNumberFormat`.

5.6 Converting Among Binary, Octal, Decimal, and Hexadecimal

Problem

You want to display an integer as a series of bits—for example, when interacting with certain hardware devices—or in some alternative number base (binary is base 2, octal is base 8, decimal is 10, hexadecimal is 16). You want to convert a binary number or a hexadecimal value into an integer.

Solution

The class `java.lang.Integer` provides the solutions. Most of the time you can use `Integer.parseInt(String input)` or `Integer.parseInt(String input, int radix)` to convert from any string containing a number to an `Integer`, and `Integer.toString(int input)` or `Integer.toString(int input, int radix)` to convert an `int` value to a `String`. Example 5-11 shows some examples of using the `Integer` class.

Example 5-11. main/src/main/java/numbers/IntegerBinOctHexEtc.java

```
String input = "101010";
int i = 42;
System.out.println(input + " in default toString() is " + Integer.toString(i));
for (int radix : new int[] { 2, 8, 10, 16, 36 }) {
  System.out.print(input + " in base " + radix + " is "
      + Integer.parseInt(input, radix) + "; ");
```

```
    System.out.println(i + " formatted in base " + radix + " is "
        + Integer.toString(i, radix));
  }
```

This program prints the binary string as an integer in various bases, and the integer 42 in those same number bases:

```
$ java numbers.IntegerBinOctHexEtc
101010 in default toString() is 42
101010 in base 2 is 42; 42 formatted in base 2 is 101010
101010 in base 8 is 33288; 42 formatted in base 8 is 52
101010 in base 10 is 101010; 42 formatted in base 10 is 42
101010 in base 16 is 1052688; 42 formatted in base 16 is 2a
101010 in base 36 is 60512868; 42 formatted in base 36 is 16
$
```

Discussion

There are also simpler versions of `toString(int)` that don't require you to specify the radix, for example, `toBinaryString()` to convert an integer to binary string representation, `toHexString()` to hexadecimal, `toOctalString()`, and so on. The Javadoc page for the `Integer` class is your friend here.

The `String` class itself includes a series of static methods—`valueOf(int)`, `valueOf(double)`, and so on—that also provide default formatting. That is, they return the given numeric value formatted as a string.

5.7 Operating on a Range of Integers

Problem

You need to work on a range of integers.

Solution

For a contiguous set, use `IntStream::range` and `rangeClosed`, or the older `for` loop.

For discontinuous ranges of numbers, use a `java.util.BitSet`. Or, just use a `List` (see Recipe 7.5).

Discussion

To process a contiguous set of integers, Java provides both `range()` / `rangeClosed()` methods in the `IntStream` and `LongStream` classes. These take a starting and ending number; `range()` excludes the ending number while `rangeClosed()` closes on, or includes, the ending number. You can also iterate over a range of numbers using the traditional `for` loop. Loop control for the `for` loop is in three parts: initialize, test, and change. If the test part is initially false, the loop will never be executed, not even once. You can iterate over the elements of an array or collection of numbers (or of any type) using a for-each loop (see Chapter 7).

The program in Example 5-12 demonstrates these techniques. The `->` syntax is a lambda expression; see Recipe 9.1 if you are not yet familiar with this syntax.

Example 5-12. main/src/main/java/numbers/NumSeries.java

```
public class NumSeries {
  public static void main(String[] args) {

    // For ordinal list of numbers n to m, use rangeClosed(start, endInclusive)
    IntStream.rangeClosed(1, 12).forEach(
      i -> System.out.println("Month # " + i));

    // Or, use a for loop starting at 1.
    for (int i = 1; i <= months.length; i++)
      System.out.println("Month # " + i);

    // Or a foreach loop
```

```
    for (String month : months) {
      System.out.println(month);
    }

    // When you want a set of array indices, use range(start, endExclusive)
    IntStream.range(0, months.length).forEach(
      i -> System.out.println("Month " + months[i]));

    // Or, use a for loop starting at 0.
    for (int i = 0; i < months.length; i++)
      System.out.println("Month " + months[i]);

    // For e.g., counting by 3 from 11 to 27, use a for loop
    for (int i = 11; i <= 27; i += 3) {
      System.out.println("i = " + i);
    }

    // A discontiguous set of integers, using a BitSet

    // Create a BitSet and turn on a couple of bits.
    BitSet b = new BitSet();
    b.set(0);  // January
    b.set(3);  // April
    b.set(8);  // September

    // Print the months
    for (int i = 0; i<months.length; i++) {
      if (b.get(i))
        System.out.println("Month " + months[i]);
    }
    // Shorter way, using IntStream stream() method
    b.stream().forEach(i->System.out.println(months[i]));

    // Same example but shorter:
    // a discontiguous set of integers, using an array
    int[] numbers = {0, 3, 8};

    // Presumably somewhere else in the code... Also a foreach loop
    for (int n : numbers) {
      System.out.println("Month: " + months[n]);
    }
  }
  /** Names of months. See Dates/Times chapter for a better way to get these */
  protected static String months[] = {
    "January", "February", "March", "April",
    "May", "June", "July", "August",
    "September", "October", "November", "December"
  };
}
```

5.8 Formatting with Correct Plurals

Problem

You're printing something like `"We used " + n + " items"`, but in English, "We used 1 items" is ungrammatical. You want "We used 1 item."

Solution

Use a `ChoiceFormat` or a conditional expression.

Discussion

Use Java's ternary operator (`cond ? trueval : falseval`) in a string concatenation. Both zero and plurals get an "s" appended to the noun in English ("no books, one book, two books"), so we test for `n==1` :

```
public class FormatPlurals {
 public static void main(String[] argv) {
  report(0);
  report(1);
  report(2);
 }

 /** report -- using conditional operator */
 public static void report(int n) {
  System.out.println("We used " + n + " item" + (n==1?"":"s"));
 }
}
```

Does it work?

```
$ java numbers.FormatPlurals
We used 0 items
We used 1 item
We used 2 items
$
```

The final `println` statement is effectively equivalent to the following:

```
if (n==1)
   System.out.println("We used " + n + " item");
else
   System.out.println("We used " + n + " items");
```

This is a lot longer, so the ternary conditional operator is worth learning.

The `ChoiceFormat` is ideal for this. It is actually capable of much more, but here I'll show only this simplest use. I specify the values 0, 1, and 2 (or more), and the string values to print corresponding to each number. The numbers are then formatted according to the range they fall into, as shown in Example 5-13.

Example 5-13. main/src/main/java/numbers/FormatPluralsChoice.java

```
public class FormatPluralsChoice extends FormatPlurals {

 // ChoiceFormat to just give pluralized word
 static double[] limits = { 0, 1, 2 };
 static String[] formats = { "reviews", "review", "reviews"};
 static ChoiceFormat pluralizedFormat = new ChoiceFormat(limits, formats);

 // ChoiceFormat to give English text version, quantified
 static ChoiceFormat quantizedFormat = new ChoiceFormat(
  "0#no reviews|1#one review|1<many reviews");

 // Test data
 static int[] data = { -1, 0, 1, 2, 3 };

 public static void main(String[] argv) {
  System.out.println("Pluralized Format");
  for (int i : data) {
   System.out.println("Found " + i + " " + pluralizedFormat.format(i));
  }

  System.out.println("Quantized Format");
  for (int i : data) {
   System.out.println("Found " + quantizedFormat.format(i));
  }
 }
}
```

This generates the same output as the basic version. It is slightly longer, but more general, and lends itself better to internationalization.

See Also

In addition to `ChoiceFormat`, the same result can be achieved with a `MessageFormat`. The online source in file *main/src/main/java/i18n/MessageFormatDemo.java* has an example.

5.9 Generating Random Numbers

Problem

You need to generate pseudorandom numbers in a hurry.

Solution

Use `java.lang.Math.random()`.

Discussion

Use `java.lang.Math.random()` to generate random numbers. There is no promise that the random values it returns are very *good* random numbers, however. Like most software-only implementations, these are *pseudorandom number generators* (PRNGs), meaning that the numbers are not totally random, but devised from an algorithm. That said, they are adequate for casual use. This code exercises the `random()` method:

```
// java.lang.Math.random( ) is static, so you don't need any constructor calls
System.out.println("A random from java.lang.Math is " + Math.random( ));
```

Note that this method only generates double values. If you need integers, construct a `java.util.Random` object and call its `nextInt()` method; if you pass it an integer value, this will become the upper bound. Here I generate a bunch of integers from 1 to 10:

```
public class RandomInt {
  public static void main(String[] a) {
    Random r = new Random();
    for (int i=0; i<1000; i++)
      // nextInt(10) goes from 0-9; add 1 for 1-10;
      System.out.println(1+r.nextInt(10));
  }
}
```

To see if this `RandomInt` demo was really working well, I used the Unix tools `sort` and `uniq`, which together give a count of how many times each value was chosen. For 1,000 integers, each of 10 values should be chosen about 100 times. I ran it twice to get a better idea of the distribution:

```
$ java numbers.RandomInt | sort | uniq -c | sort -k 2 -n
  96 1
 107 2
 102 3
 122 4
  99 5
 105 6
  97 7
  96 8
  79 9
  97 10
$ java numbers.RandomInt | sort | uniq -c | sort -k 2 -n
  86 1
  88 2
 110 3
  97 4
```

```
 99 5
109 6
 82 7
116 8
 99 9
114 10
$
```

Not bad, but the next step is to run these through a statistical program to see how random they really are; we'll return to this in a minute.

In general, to generate random numbers, you need to construct a `java.util.Random` object (not just any old random object) and call its `next*()` methods. These methods include `nextBoolean()`, `nextBytes()` (which fills the given array of bytes with random values), `nextDouble()`, `nextFloat()`, `nextInt()`, and `nextLong()`. Don't be confused by the capitalization of `Float`, `Double`, etc. They return the primitive types `boolean`, `float`, `double`, etc., not the capitalized wrapper objects. Clear enough? Maybe an example will help:

```
// java.util.Random methods are non-static; we need an instance
Random r = new Random();
for (int i=0; i<10; i++) {
  System.out.println("A double from java.util.Random is " + r.nextDouble());
}
for (int i=0; i<10; i++) {
  System.out.println("An integer from java.util.Random is " + r.nextInt());
}
```

A fixed value (*starting seed*) can be provided to generate repeatable values, as for testing. You can also use the `java.util.Random nextGaussian()` method, as shown next. The `nextDouble()` methods try to give a flat distribution between 0 and 1.0, in which each value has an equal chance of being selected. A Gaussian or normal distribution is a bell curve of values from negative infinity to positive infinity, with the majority of the values around zero (0.0):

```
// numbers/Random3.java
Random r = new Random();
for (int i = 0; i < 10; i++)
  System.out.println("A gaussian random double is " + r.nextGaussian());
```

To illustrate the different distributions, I generated 10,000 numbers first by using `nextRandom()` and then using `nextGaussian()`. The code for this is in *Random4.java* (not shown here) and is a combination of the previous programs with code to print the results into files. I then plotted histograms using the R statistics package (see Chapter 12 and the R Project website). The R script used to generate the graph, *randomnesshistograms.r*, doesn't contain any Java code; it's in *javasrc* under *main/src/main/resources*. The results are shown in Figure 5-1.

The bar graph should be reasonably flat, and the Gaussian should resemble the hated-by-students "bell curve." Looks like both PRNGs do their job!

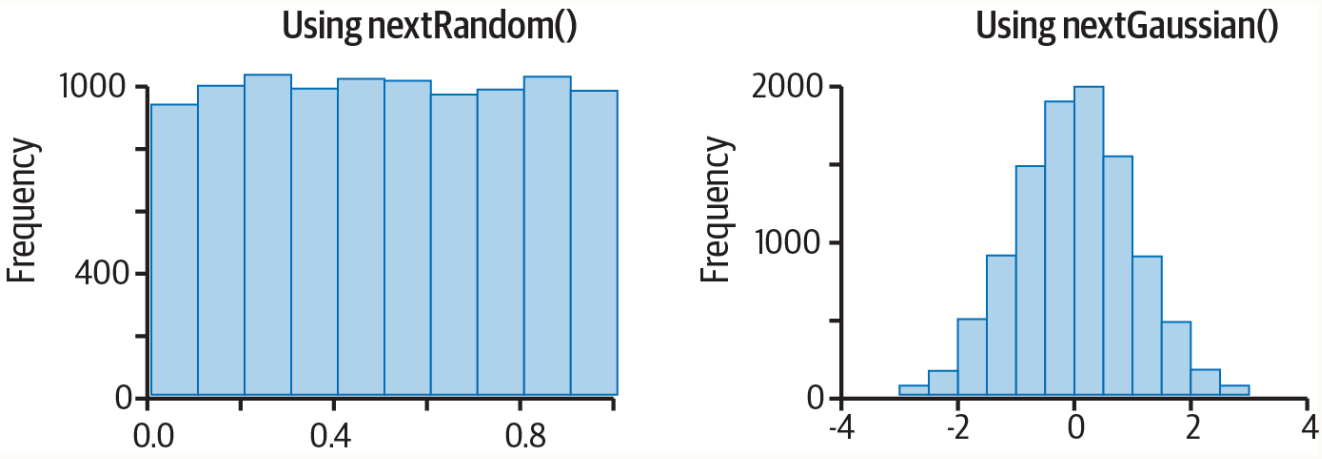


Figure 5-1. Flat (left) and Gaussian (right) distributions

See Also

For more information, see the Javadoc documentation for `java.util.Random`, and the discussion in the introdcution to Chapter 5 about pseudorandomness versus real randomness. This is also treated in my *Java Magazine* writeup on random numbers, where I give an example of "true" randomness via 1980s hardware.

For cryptographic use, see the class `java.security.SecureRandom`, which provides cryptographically strong pseudorandom number generators.

5.10 Multiplying Matrices

Problem

You need to multiply a pair of two-dimensional arrays, as is common in mathematical and engineering applications.

Solution

Use the following code with nested loops as a model.

Discussion

It is straightforward to multiply an array of a numeric type. In real life you would probably use a full-blown package such as the Efficient Java Matrix Library (EJML) or Deeplearning4j's ND4J package. However, a simple implementation can serve to show the concepts involved. The code in Example 5-14 implements matrix multiplication.

Example 5-14. main/src/main/java/numbers/Matrix.java

```
public class Matrix {

  /* Matrix-multiply two arrays together.
   * The arrays MUST be rectangular.
   * @author Adapted from Tom Christiansen & Nathan Torkington's
   * implementation in their Perl Cookbook.
   */
  public static int[][] multiply(int[][] m1, int[][] m2) {
    int m1rows = m1.length;
    int m1cols = m1[0].length;
    int m2rows = m2.length;
    int m2cols = m2[0].length;
    if (m1cols != m2rows)
      throw new IllegalArgumentException(
        "matrices don't match: " + m1cols + " != " + m2rows);
    int[][] result = new int[m1rows][m2cols];

    // multiply
    for (int i=0; i<m1rows; i++) {
      for (int j=0; j<m2cols; j++) {
        for (int k=0; k<m1cols; k++) {
          result[i][j] += m1[i][k] * m2[k][j];
        }
      }
    }

    return result;
  }

  /** Matrix print.
   */
  public static void mprint(int[][] a) {
    int rows = a.length;
    int cols = a[0].length;
    System.out.println("array["+rows+"]["+cols+"] = {");
    for (int i=0; i<rows; i++) {
      System.out.print("{");
```

```
      for (int j=0; j<cols; j++)
        System.out.print(" " + a[i][j] + ",");
      System.out.println("},");
    }
    System.out.println("};");
  }
}
```

Here is a program (in the same source directory) that uses the `Matrix` class to multiply two arrays of `int`s:

```
    int x[][] = {
      { 3, 2, 3 },
      { 5, 9, 8 },
    };
    int y[][] = {
      { 4, 7 },
      { 9, 3 },
      { 8, 1 },
    };
    int z[][] = Matrix.multiply(x, y);
    Matrix.mprint(x);
    Matrix.mprint(y);
    Matrix.mprint(z);
```

See Also

Consult a book on numerical methods for more things to do with matrices; one of our reviewers recommends the series of *Numerical Recipes* books by William Press et al. (Cambridge University Press), available from the Numerical Recipes website. There are several translations of the book's code into various languages, including Java. Pricing varies by package; the book and the accompanying software are priced separately.

There are some commercial software packages that can do some of these calculations for you. Do a web search to find the current offerings.

Note: for single-dimension arrays (e.g., vectors), you can just use the Vector API described in Recipe 5.11.

5.11 Optimizing Large Arithmetic Operations with Vector Operations 22C

Problem

You need to perform arithmetic operations on a large collection of numbers.

Solution

Use the Vector arithmetic package.

Discussion

Most modern computer processors (CPUs) include machine instructions for performing vector calculations in hardware, faster than one could perform by writing code in software. These instructions will operate on two vectors (*arrays* in Java terms), and take a set of numbers (usually four, eight, or some multiple) of numbers from each, and perform the given operation (add, multiply, etc.) on all eight numbers from each of the two vectors, *in a single machine cycle*, thus significantly speeding things up. This is called Single Instruction Multiple Data, or SIMD, and it is one of the *S* letters in Streaming SIMD Extensions (SSE). SSE is one of several vector instruction sets, including:

- SSE and Advanced Vector Extensions (AVX) on amd64/Intel 64-bit architectures
- Neon and Scalable Vector Extensions (SVE) on ARM AArch64 architectures

Naturally, chip makers such as AMD and Intel try to be compatible with each other while extending the CPU features to try to out-do each other. Java's vector operations support tries to do the best it can to take advantage of these features, while providing equally accurate software implementation of operations that are not supported on a given machine.

The Vector API is in incubation status as of Java 22, so you need to pass the `--add-module=jdk.incubator.vector` and import from that package when using this API in software.

While Java already has some auto-vectorization, it is hard to predict when it will kick in. The Vector API makes the vectorization explicit, and is intended to work on any platform, using SIMD instructions if available and falling back to a software implementation if SIMD is not available.

There is a series of specialized terminology used here. Elements within a Vector are called *lanes*, as they can run in parallel, like the lanes of a highway. The *shape* of a Vector is how many bits it can process at a time. `Vector<E>` is abstract; there exists a concrete Vector subclass for each of Java's six numeric types (excluding `char`): `ByteVector`, `ShortVector`, `IntVector`, `LongVector`, `FloatVector`, and `DoubleVector`. The combination of data type and shape is called a *species*.

In using the API, one has to choose a *species* of vectorization, or just let the system choose by using one labeled `PREFERRED`. The species will usually be stored in a `static final` variable for performance reasons. The array data has to be wrapped in one of the six Vector subclasses. Then the operations are specified, using method names rather than syntax operators.

The code in Example 5-15 shows an example of a simple vector calculation of the formula $ax^2 + 2b$ on each pair of numbers taken from two parallel arrays of `double`s. The calculation is done on a small array of numbers, both using the Vector API and performing the (reasonably simple) calculation explicitly.

Example 5-15. incubation/src/main/java/numbers/VectorOps.java—Demo of Vector API

```
import jdk.incubator.vector.DoubleVector;
import jdk.incubator.vector.VectorSpecies;

/** Compute ax^2+2b on double arrays */
public class VectorOps {

 static final VectorSpecies<Double> SPECIES = DoubleVector.SPECIES_PREFERRED;

 public static void main(String[] args) {
  System.out.println("Using Vector API with Species " + SPECIES);
  double[] a = { 2, 4, 6, 8, 9, 10, 11, 12, 13, 14};
  double x = Math.PI;
  double[] b = { 1, 3, 5, 7, 9, 10, 11, 12, 13, 14};
  double[] sresults = new double[a.length], vresults = new double[a.length];
  System.out.println(
   "Inputs: a=" + Arrays.toString(a) +
    ", x = " + x + ", b = " + Arrays.toString(b));

  scalarComputation(a, x, b, sresults);
  System.out.println("Results from scalar = " +
   Arrays.toString(sresults) + ")");

  vectorComputation(a, x, b, vresults);
  System.out.println("Results from vector = " +
   Arrays.toString(vresults) + ")");

  for (int i = 0; i < sresults.length; i++) {
   if (Math.abs(sresults[i] - vresults[i]) > 0.00000001D) {
    throw new IllegalStateException(
     "Values differ (" +
     sresults[i] + " != " + vresults[i] + ")");
   }
  }
  System.out.println("Computed both ways: Close enough!");
 }

 static void scalarComputation(double[] a, double x, double[] b, double[] c) {
  for (int i = 0; i < a.length; i++) {
   c[i] = (a[i] * x * x + b[i] * 2);
  }
```

```
  }

  static void vectorComputation(double[] a, double x, double[] b, double[] c) {
    int i = 0;
    int upperBound = SPECIES.loopBound(a.length);
    for (; i < upperBound; i += SPECIES.length()) {
      var va = DoubleVector.fromArray(SPECIES, a, i);
      var vb = DoubleVector.fromArray(SPECIES, b, i);
      System.out.println("va length = " + va.length() + ")");
      var vc = va
         .mul(x*x)
         .add(vb.mul(2));
      vc.intoArray(c, i);
    }
    // Handle leftover (iff input length % SPECIES.length() != 0)
    for (; i < a.length; i++) {
      c[i] = (a[i] * x * x + b[i] * 2);
    }
  }
}
```

The simplistic implementation is clearly shorter, but the Vector implementation will be faster, though it's not easy to measure the difference on such a small dataset. The spread will almost certainly increase as the dataset size increases. The answers are almost the same, except that one differs in the 14th digit (compare the last digit of the sixth number in each result). This tiny difference shows that the Vector and software solutions provide the "same" answer, while confirming that different code was used to produce the two outputs.

Running the program produces the following output. The `scalar` results are from the simplistic implementation, and the `vector` results are from the Vector API:

```
$ java --add-modules jdk.incubator.vector \
incubation/src/main/java/numbers/VectorOps.java
WARNING: Using incubator modules: jdk.incubator.vector
Note: ../incubation/src/main/java/numbers/VectorOps.java uses preview features of Java SE 21.
Note: Recompile with -Xlint:preview for details.
Using Vector API with Species Species[double, 4, S_256_BIT]
Inputs: a=[2.0, 4.0, 6.0, 8.0, 9.0, 10.0, 11.0, 12.0, 13.0, 14.0], x = 3.141592653589793, b = [1.0, 3.0, 5.0, 7.0, 9.0, 10.0, 11.0, 12.0, 13.0, 14.0]
Results from scalar = [21.739208802178716, 45.47841760435743, 69.21762640653614, 92.95683520871486, 106.82643960980423, 118.69604401
va length = 4
va length = 4
Results from vector = [21.739208802178716, 45.47841760435743, 69.21762640653614, 92.95683520871486, 106.82643960980423, 118.6960440
Computed both ways: Close enough!
$
```

As can be seen from the species printout, this species of computation takes four elements at a time. Thus the printout "va length = 4" occurs twice, taking 8 lanes of the 10 in the vector. The remaining two are handled by the extra loop at the end of `vectorComputation()`.

The `Vector` class features quite a few other operators. The basic arithmetic ones are `add()`, `sub()`, `mul()`, `div()`, `abs()`, `min()`, `max()`, `neg()`, `eq()`, `lt()` (but no `gt()`). There are many, many methods for reshaping, extracting, and slicing and dicing the vectors. The best reference is the Javadoc page for the `Vector` class. Although it's in incubation, this man page is included in the standard Javadoc set.

5.12 Using Complex Numbers

Problem

You need to manipulate complex numbers, as is common in mathematical, scientific, or engineering applications.

Solution

Find, or write, a class to keep track of the real and imaginary parts of a complex number.

Discussion

Java does not provide any explicit support for dealing with complex numbers. You could keep track of the real and imaginary parts and do the computations yourself, but that is not a very well-structured solution.

A better solution, of course, is to use a class that implements complex numbers. I have written such a class, but now I recommend using the `Complex` class in the Apache Commons Math library, which is more complete. The build coordinates for this are `org.apache.commons:commons-math3:3.6.1` (or later). First, an example of using Apache's library:

```
public class ComplexDemoACM {

  public static void main(String[] args) {
    Complex c = new Complex(3,  5);
    Complex d = new Complex(2, -2);
    System.out.println(c);
    System.out.println(c + ".getReal() = " + c.getReal());
    System.out.println(c + " + " + d + " = " + c.add(d));
    System.out.println(c + " * " + d + " = " + c.multiply(d));
    System.out.println(c.divide(d));
  }
}
```

Running this demo program produces the following output:

```
(3.0, 5.0)
(3.0, 5.0).getReal() = 3.0
(3.0, 5.0) + (2.0, -2.0) = (5.0, 3.0)
(3.0, 5.0) * (2.0, -2.0) = (16.0, 4.0)
(-0.5, 2.0)
```

Example 5-16 is the source for my version of the `Complex` class and shouldn't require much explanation. The Apache class is admittedly more sophisticated, but I leave mine here just to demystify the basic operation of complex numbers.

To keep the API general, I provide—for each of add, subtract, and multiply—both a static method that works on two complex objects and a nonstatic method that applies the operation to the given object and one other object.

Example 5-16. main/src/main/java/numbers/Complex.java

```
public record Complex(double r, double i) {

  /** Display the current Complex as a String, for use in
   * println() and elsewhere.
   */
  public String toString() {
    StringBuilder sb = new StringBuilder().append(r);
    if (i>0)
      sb.append('+');  // else append(i) appends - sign
    return sb.append(i).append('i').toString();
  }

  /** Return just the Real part */
  public double getReal() {
    return r;
  }
  /** Return just the Real part */
  public double getImaginary() {
    return i;
  }
  /** Return the magnitude of a complex number */
  public double magnitude() {
    return Math.sqrt(r*r + i*i);
  }

  /** Add another Complex to this one
```

```
   */
  public Complex add(Complex other) {
    return add(this, other);
  }

  /** Add two Complexes
   */
  public static Complex add(Complex c1, Complex c2) {
    return new Complex(c1.r+c2.r, c1.i+c2.i);
  }

  /** Subtract another Complex from this one
   */
  public Complex subtract(Complex other) {
    return subtract(this, other);
  }

  /** Subtract two Complexes
   */
  public static Complex subtract(Complex c1, Complex c2) {
    return new Complex(c1.r-c2.r, c1.i-c2.i);
  }

  /** Multiply this Complex times another one
   */
  public Complex multiply(Complex other) {
    return multiply(this, other);
  }

  /** Multiply two Complexes
   */
  public static Complex multiply(Complex c1, Complex c2) {
    return new Complex(c1.r*c2.r - c1.i*c2.i, c1.r*c2.i + c1.i*c2.r);
  }

  /** Divide c1 by c2.
   * @author Gisbert Selke.
   */
  public static Complex divide(Complex c1, Complex c2) {
    return new Complex(
      (c1.r*c2.r+c1.i*c2.i)/(c2.r*c2.r+c2.i*c2.i),
      (c1.i*c2.r-c1.r*c2.i)/(c2.r*c2.r+c2.i*c2.i));
    }
}
```

Running the simple demo file `ComplexDemoIan` in the same directory will show this in action:

```
3.0+5.0i
3.0+5.0i.getReal() = 3.0
3.0+5.0i + 2.0-2.0i = 5.0+3.0i
3.0+5.0i + 2.0-2.0i = 5.0+3.0i
3.0+5.0i * 2.0-2.0i = 16.0+4.0i
3.0+5.0i / 2.0-2.0i = -0.5+2.0i
```

5.13 Handling Very Large Numbers

Problem

You need to handle integer numbers larger than `Long.MAX_VALUE` or floating-point values larger than `Double.MAX_VALUE`.

Solution

Use the `BigInteger` or `BigDecimal` values in package java.math, as shown in Example 5-17.

Example 5-17. main/src/main/java/numbers/BigNums.java

```
System.out.println("Here's Long.MAX_VALUE: " + Long.MAX_VALUE);
BigInteger bInt = new BigInteger("34192292233720368547758O7");
System.out.println("Here's a bigger number: " + bInt);
System.out.println("Here it is as a double: " + bInt.doubleValue());
```

Note that the constructor takes the number as a string. Obviously you couldn't just type the numeric digits because, by definition, these classes are designed to represent numbers larger than will fit in a Java `long`.

Discussion

Both `BigInteger` and `BigDecimal` objects are immutable; that is, once constructed, they always represent a given number. That said, a number of methods return new objects that are mutations of the original, such as `negate()`, which returns the negative of the given `BigInteger` or `BigDecimal`. There are also methods corresponding to most of the Java language built-in operators defined on the base types `int` / `long` and `float` / `double`. The division method makes you specify the rounding method; consult a book on numerical analysis for details. Example 5-18 is a simple stack-based calculator using `BigDecimal` as its numeric data type.

Example 5-18. main/src/main/java/numbers/BigNumCalc.java

```
public class BigNumCalc {

 /** an array of Objects, simulating user input */
 public static Object[] testInput = {
  new BigDecimal("34192292233720368547758O7.23343"),
  new BigDecimal("2.0"),
  "*",
 };

 public static void main(String[] args) {
  BigNumCalc calc = new BigNumCalc();
  System.out.println(calc.calculate(testInput));
 }

 /**
  * Stack of numbers being used in the calculator.
  */
 Stack<BigDecimal> stack = new Stack<>();

 /**
  * Calculate a set of operands; the input is an Object array containing
  * either BigDecimal objects (which may be pushed onto the Stack) and
  * operators (which are operated on immediately).
  * @param input
  * @return
  */
 public BigDecimal calculate(Object[] input) {
  BigDecimal tmp;
  for (int i = 0; i < input.length; i++) {
   Object o = input[i];
   if (o instanceof BigDecimal decimal) {
    stack.push(decimal);
   } else if (o instanceof String string) {
    switch (string.charAt(0)) {
    // + and * are commutative, order doesn't matter
    case '+':
     stack.push((stack.pop()).add(stack.pop()));
     break;
    case '*':
     stack.push((stack.pop()).multiply(stack.pop()));
     break;
    // - and /, order *does* matter
    case '-':
     tmp = stack.pop();
     stack.push((stack.pop()).subtract(tmp));
     break;
```

```
      case '/':
        tmp = stack.pop();
        stack.push((stack.pop()).divide(tmp, RoundingMode.HALF_UP));
        break;
      default:
        throw new IllegalStateException("Unknown OPERATOR popped");
      }
    } else {
      throw new IllegalArgumentException("Syntax error in input");
    }
   }
   return stack.pop();
  }
}
```

Running this produces the expected (very large) value:

```
$ java numbers/BigNumCalc.java
6838458446744073709551614.466860
$
```

The current version has its inputs hardcoded, as does the JUnit test program, but in real life you can use regular expressions to extract words or operators from an input stream (as in Recipe 4.6), or you can use the `StreamTokenizer` approach of the simple calculator (see Recipe 10.7). The stack of numbers is maintained using a `java.util.Stack` (see Recipe 7.6).

`BigInteger` is mainly useful in cryptographic and security applications. Its method `isProbablyPrime()` can create prime pairs for public key cryptography. `BigDecimal` might also be useful in computing the size of the universe.

5.14 Program: TempConverter

The program shown in Example 5-19 prints a table of Fahrenheit temperatures (still used in daily weather reporting in a few countries like the US and its territories, Liberia, and some countries in the Caribbean) and the corresponding Celsius temperatures (used in science everywhere and in daily life in most of the world).

Example 5-19. main/src/main/java/numbers/TempConverter.java

```
public class TempConverter {

  public static void main(String[] args) {
    TempConverter t = new TempConverter();
    t.start();
    t.data();
    t.end();
  }

  protected void start() {
  }

  protected void data() {
    for (int i=-40; i<=120; i+=10) {
      double c = fToC(i);
      print(i, c);
    }
  }

  public static double cToF(double deg) {
    return ( deg * 9 / 5) + 32;
  }

  public static double fToC(double deg) {
    return ( deg - 32 ) * ( 5d / 9 );
  }

  protected void print(double f, double c) {
```

```
    System.out.println(f + " " + c);
  }

  protected void end() {
  }
}
```

This works, but these numbers print with about 15 digits of (useless) decimal fractions! The second version of this program subclasses the first and uses `printf` (see Recipe 3.2) to control the formatting of the converted temperatures (see Example 5-20). It will now look much better, assuming you're printing in a monospaced font.

Example 5-20. main/src/main/java/numbers/TempConverter2.java

```
public class TempConverter2 extends TempConverter {

  public static final String SEP = "-----------------";

  public static void main(String[] args) {
    TempConverter t = new TempConverter2();
    t.start();
    t.data();
    t.end();
  }

  @Override
  protected void print(double f, double c) {
    System.out.printf("%6.2f\u00B0F %6.2f\u00B0C%n", f, c);
  }

  @Override
  protected void start() {
    System.out.println("Fahr     Celsius");
    System.out.println(SEP);
  }

  @Override
  protected void end() {
    System.out.println(SEP);
  }
}
```

Here is the output of running this program:

```
$ java numbers.TempConverter2
Fahr     Celsius
-----------------
-40.00°F -40.00°C
-30.00°F -34.44°C
-20.00°F -28.89°C
-10.00°F -23.33°C
  0.00°F -17.78°C
 10.00°F -12.22°C
 20.00°F  -6.67°C
 30.00°F  -1.11°C
 40.00°F   4.44°C
 50.00°F  10.00°C
 60.00°F  15.56°C
 70.00°F  21.11°C
 80.00°F  26.67°C
 90.00°F  32.22°C
100.00°F  37.78°C
110.00°F  43.33°C
120.00°F  48.89°C
-----------------
$
```

1 For a low-cost source of randomness, check out the now-defunct Lavarand. The process used digitized video of 1970s lava lamps to provide "hardware-based" randomness. Fun!

2 Note that an expression consisting entirely of compile-time constants, like `Math.PI * 2.1e17`, was also considered to be Strict-FP.

Previous chapter
< 4. String Matching with Regular Expressions

Next chapter
6. Dates and Times >